

Part 1

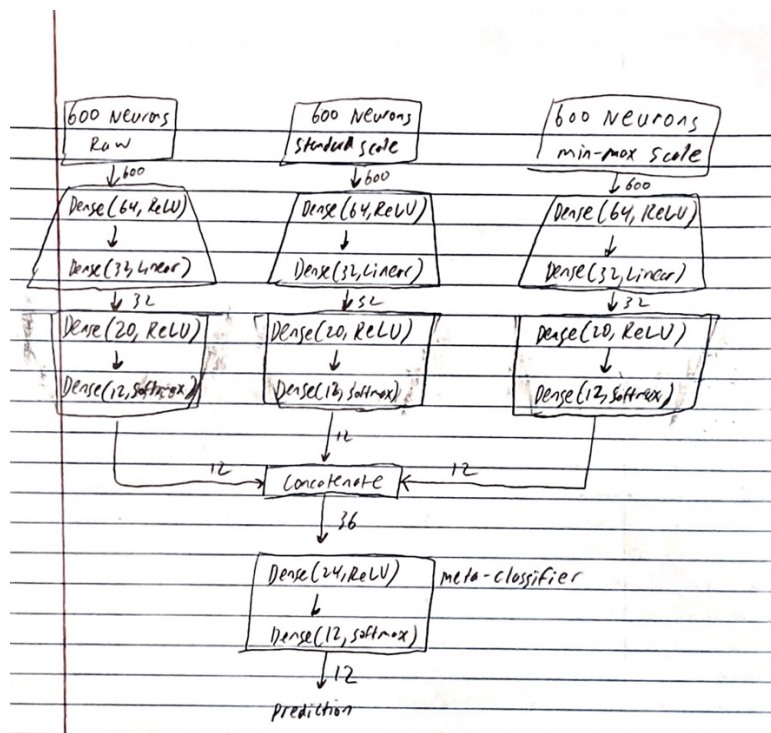
Link: <https://studio.edgeimpulse.com/public/1006236/live>

```
Timing: DSP 84 ms, inference 578 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
Idle: 0.863281
Lift: 0.136719
```

```
Timing: DSP 84 ms, inference 552 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
Idle: 0.000000
Lift: 0.996094
```

Part 2

1.



2.

Pruning is applied with `prune_low_magnitude()`. Using polynomial decay, 10% of the weights are set to zero and that ramps up to 80% of the weights being zero by step 500. It's important to strip the pruning wrappers before QAT so that the extra metadata is removed and a clean Keras model remains. The Keras model can then be fine-tuned with QAT. QAT is applied after pruning by rounding float32 weights and activations to approximate the int8 versions, but are still kept as float32. The model fine-tunes on this fake quantization for 5 epochs. The mask enforcement step is used during and after QAT because weights that were pruned to zero need to stay as zero. If there was no mask enforcement, then QAT could shift some weights away from zero, which loses sparsity in the model. The final model is converted to int8 by

Nick Leung EE446 SP26 Lab 8 Report

using a converter, changing its supported ops to int8, and setting the input and output types to int8. Representative data is needed so the converter knows what the minimum and maximum values are for each activation layer. Pruning and int8 quantization helps when deploying to a resource-constrained device by making the model less complex and easier to compress. Quantization reduces the model size by a factor of 4 (float32 to int8). This reduces the memory resources consumed by the model on the device.

3.

```
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.9648, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0312
Standard-scaled model scores: 0.0000, 0.8672, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0312, 0.0039, 0.0937
Min-max-scaled model scores: 0.0000, 0.0078, 0.1445, 0.0039, 0.7461, 0.0195, 0.0742, 0.0000, 0.0000, 0.0039, 0.0000, 0.0000
Meta-classifier scores: 0.0117, 0.0000, 0.0000, 0.0000, 0.7344, 0.0039, 0.1484, 0.0469, 0.0078, 0.0234, 0.0234, 0.0000
-----
Final prediction: Climbing stairs
Class index: 4
Confidence score: 0.7344
-----
```

This is not acceptable for the resting board condition because the model thinks the IMU is moving (movement is required to climb stairs) when it in fact isn't. Each of the branches has a different majority prediction. Raw predicts lying down, standard predicts sitting and relaxing, and min-max predicts climbing stairs. The meta-classifier agrees the most with the min-max model. There could also be a mismatch with the training data and real board orientation. When I changed orientations and hit reset, the first prediction was jogging instead of climbing stairs. One change to improve reliability could be including a class with data for a resting board if someone took the sensor off. Another change could be taking multiple windows of samples and then taking a majority vote.

4.

Sometimes the model predicted the same class when the board was moved differently. When I rotate the board by 180 degrees along one axis, the board keeps predicting climbing stairs. On another axis, it keeps predicting frontal elevation of arms. This could be explained by the sensor being in a different orientation when collecting training data compared to it sitting flat on a table. This would cause different axes to have different accelerations than expected. The ranges for standard and min-max models could be different than the Arduino's range of IMU readings, which could make the models unreliable.